

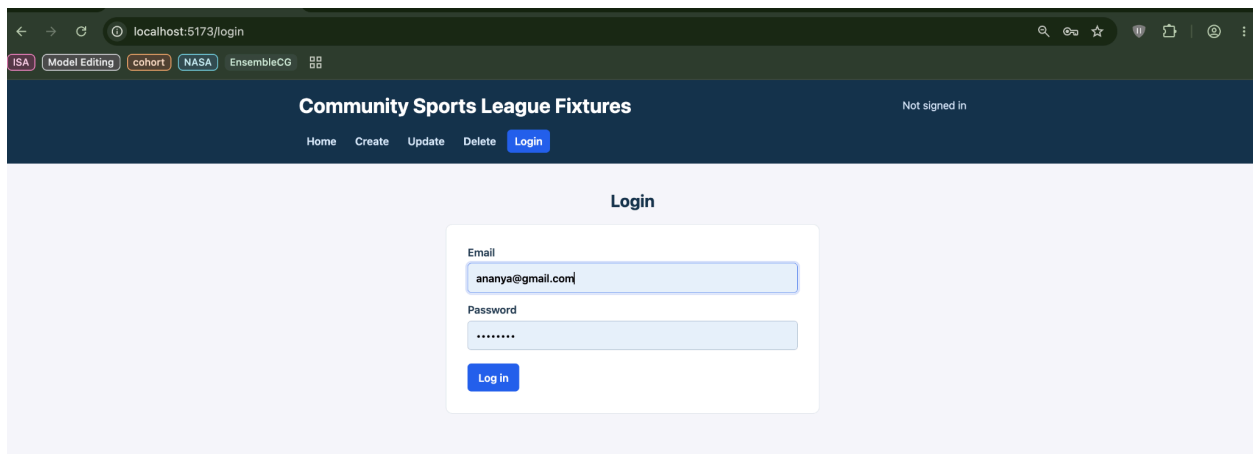
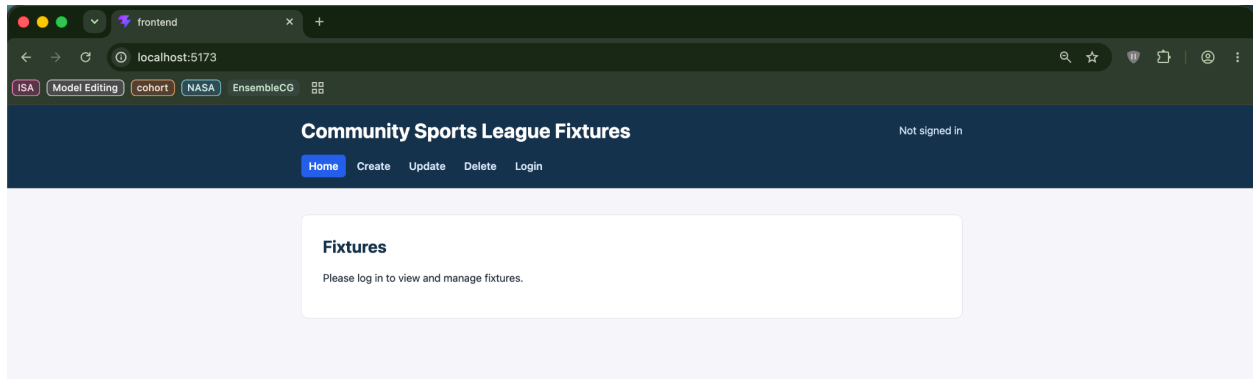
Homework 4 Report

Section 0 - Personal Configuration and Domain

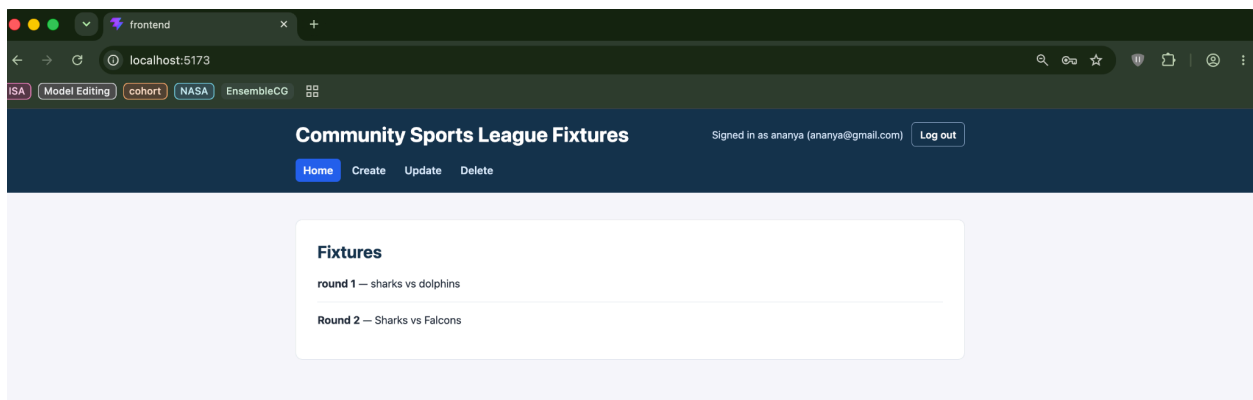
Value	Definition
SID4	2383
PORT_BASE	8583
PREFIX	s2383
SEED	2383
VERIFY_SEED	262383
DOMAIN_ID	7
Hardware	MacBook Pro, Apple M4 (10-core CPU), 16 GB RAM
Python version	3.11.9
Tagged commit hash	

Part 1 - React Client

1. Login and Home



(Some random data inserted just to test home)



Code:

```
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [error, setError] = useState("");
const [isSubmitting, setIsSubmitting] = useState(false);

async function handleSubmit(event) {
  event.preventDefault();

  setError("");
  setIsSubmitting(true);

  try {
    const response = await fetch("/api/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      credentials: "include", //allows the browser to
      body: JSON.stringify({
        email,
        password,
      }),
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.detail || "Login failed");
    }
    // onLogin={setUser} passes the parent state-update
    onLogin({
      name: data.name,
      email: data.email,
    });

    navigate("/");
  } catch (requestError) {
    setError(requestError.message);
  } finally {
    setIsSubmitting(false);
  }
}
```

```
return (
  <section>
    <h2>Login</h2>
    { /* handleSubmit prevents the browser's normal form reload and sets state */ }
    <form onSubmit={handleSubmit}>
      <label>
        Email
        <input
          type="email"
          value={email}
          onChange={(event) => setEmail(event.target.value)}
          required
        />
      </label>

      <label>
        Password
        <input
          type="password"
          value={password}
          onChange={(event) => setPassword(event.target.value)}
          required
        />
      </label>

      {error && <p role="alert">{error}</p>}

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? "Logging in..." : "Log in"}
      </button>
    </form>
  </section>
);
```

2. Add a New Record

The screenshot shows a web browser window with the URL `localhost:5173/create`. The browser's address bar and tabs are visible. The application's header is dark blue with the title "Community Sports League Fixtures" and a user login status "Signed in as ananya (ananya@gmail.com)" with a "Log out" button. Below the header is a navigation bar with links: "Home", "Create" (highlighted in blue), "Update", and "Delete". The main content area is light blue and contains a white box titled "Create Fixture". Inside this box are three text input fields: "Fixture name" (containing "Round 3"), "Team one" (containing "Falcons"), and "Team two" (containing "Dolphins"). At the bottom of the box is a blue button labeled "Create fixture".

The screenshot shows the same web browser window, but the URL is `localhost:5173`. The application's header and navigation bar are identical to the previous screenshot. The main content area is light blue and contains a white box titled "Fixtures". Inside this box is a table with three rows of fixtures.

Fixture name	Team one	Team two
round 1 — sharks vs dolphins		
Round 2 — Sharks vs Falcons		
Round 3 — Falcons vs Dolphins		

Code:

```
export default function CreateRecord({ user }) { //user is passed from App as a prop
  const navigate = useNavigate();

  const [fixtureName, setFixtureName] = useState("");
  const [teamOne, setTeamOne] = useState("");
  const [teamTwo, setTeamTwo] = useState("");
  const [error, setError] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);

  async function handleSubmit(event) {
    event.preventDefault();

    setError("");
    setIsSubmitting(true);

    try {
      const response = await fetch("/api/fixtures", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        credentials: "include",
        body: JSON.stringify({
          fixture_name: fixtureName,
          teams: `${teamOne} vs ${teamTwo}`,
        }),
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.detail || "Could not create fixture");
      }

      navigate("/");
    } catch (requestError) {
      setError(requestError.message);
    } finally {
      setIsSubmitting(false);
    }
  }
}
```

```
if (!user) {
  return (
    <section>
      <h2>Create Fixture</h2>
      <p>Please log in before creating a fixture.</p>
    </section>
  );
}

return (
  <section>
    <h2>Create Fixture</h2>

    <form onSubmit={handleSubmit}>
      <label>
        Fixture name
        <input
          type="text"
          value={fixtureName}
          onChange={(event) => setFixtureName(event.target.value)}
          required
        />
      </label>

      <label>
        Team one
        <input
          type="text"
          value={teamOne}
          onChange={(event) => setTeamOne(event.target.value)}
          required
        />
      </label>

      <label>
        Team two
        <input
          type="text"
          value={teamTwo}
          onChange={(event) => setTeamTwo(event.target.value)}
          required
        />
      </label>
    </form>
  </section>
);
```

```
    <button type="submit" disabled={isSubmitting}>
      {isSubmitting ? "Creating..." : "Create fixture"}
    </button>
  </form>
</section>
);
}
```

3. Update Record

The screenshot shows a web browser window with the URL `localhost:5173/update`. The application header is dark blue with the title "Community Sports League Fixtures" and a user login status "Signed in as ananya (ananya@gmail.com)" with a "Log out" button. Below the header is a navigation bar with links: "Home", "Create", "Update" (highlighted in blue), and "Delete".

The main content area is light purple and features a white box titled "Update Fixture". Inside this box, there are four input fields:

- "Select fixture" is a dropdown menu currently showing "round 1".
- "Fixture name" is a text input field containing "Semi-final".
- "Team one" is a text input field containing "sharks".
- "Team two" is a text input field containing "dolphins".

At the bottom of the white box is a blue button labeled "Update fixture".

The screenshot shows the same web browser window, but the URL is `localhost:5173`. The application header and navigation bar are identical to the previous screenshot. The main content area is light purple and features a white box titled "Fixtures".

Inside the "Fixtures" box is a table with the following data:

Fixture
Semi-final — sharks vs dolphins
Round 2 — Sharks vs Falcons
Round 3 — Falcons vs Dolphins

Code:

```
export default function UpdateRecord({ user }) {
  const navigate = useNavigate();

  const [fixtures, setFixtures] = useState([]);
  const [selectedFixtureId, setSelectedFixtureId] = useState("");
  const [fixtureName, setFixtureName] = useState("");
  const [teamOne, setTeamOne] = useState("");
  const [teamTwo, setTeamTwo] = useState("");
  const [error, setError] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);

  useEffect(() => { // fetches the protected fixture list once a user is avail
    if (!user) {
      return;
    }

    async function loadFixtures() {
      setError("");

      try {
        const response = await fetch("/api/fixtures", {
          credentials: "include",
        });

        const data = await response.json();

        if (!response.ok) {
          throw new Error(data.detail || "Could not load fixtures");
        }

        setFixtures(data);
      } catch (requestError) {
        setError(requestError.message);
      }
    }

    loadFixtures();
  }, [user]);
```

```
async function handleSubmit(event) {
  event.preventDefault();

  if (!selectedFixtureId) {
    setError("Select a fixture to update.");
    return;
  }

  setError("");
  setIsSubmitting(true);

  try {
    const response = await fetch(
      `/api/fixtures/${selectedFixtureId}`,
      {
        method: "PUT",
        headers: {
          "Content-Type": "application/json",
        },
        credentials: "include",
        body: JSON.stringify({
          fixture_name: fixtureName,
          teams: `${teamOne} vs ${teamTwo}`,
        }),
      }
    );

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.detail || "Could not update fixture");
    }

    navigate("/");
  } catch (requestError) {
    setError(requestError.message);
  } finally {
    setIsSubmitting(false);
  }
}
```

```
function handleFixtureChange(event) {
  const fixtureId = Number(event.target.value);

  setSelectedFixtureId(fixtureId);

  const selectedFixture = fixtures.find(
    (fixture) => fixture.id === fixtureId,
  );

  if (!selectedFixture) {
    setFixtureName("");
    setTeamOne("");
    setTeamTwo("");
    return;
  }

  const teams = splitTeams(selectedFixture.teams);

  setFixtureName(selectedFixture.fixture_name);
  setTeamOne(teams.teamOne);
  setTeamTwo(teams.teamTwo);
}
```

```
if (!user) {
  return (
    <section>
      <h2>Update Fixture</h2>
      <p>Please log in before updating a fixture.</p>
    </section>
  );
}

return (
  <section>
    <h2>Update Fixture</h2>

    {isLoading && <p>Loading fixtures...</p>}

    {isLoading && (
      <form onSubmit={handleSubmit}>
        <label>
          Select fixture
          <select /* chooses which fixture to edit */
            value={selectedFixtureId}
            onChange={handleFixtureChange}
            required
          >
            <option value="">Choose a fixture</option>

            {fixtures.map((fixture) => (
              <option key={fixture.id} value={fixture.id}>
                {fixture.fixture_name}
              </option>
            ))}
          </select>
        </label>
      </form>
    )}
```

```

<label>
  Fixture name
  <input
    type="text"
    value={fixtureName}
    onChange={(event) => setFixtureName(event.target.value)}
    required
  />
</label>

<label>
  Team one
  <input
    type="text"
    value={teamOne}
    onChange={(event) => setTeamOne(event.target.value)}
    required
  />
</label>

<label>
  Team two
  <input
    type="text"
    value={teamTwo}
    onChange={(event) => setTeamTwo(event.target.value)}
    required
  />
</label>

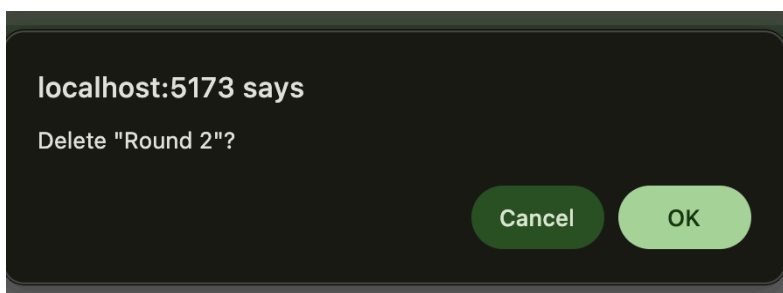
{error && <p role="alert">{error}</p>}

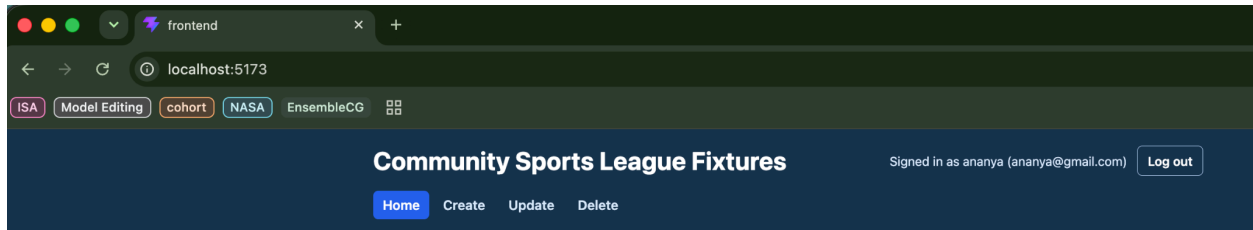
<button type="submit" disabled={isSubmitting}>
  {isSubmitting ? "Updating..." : "Update fixture"}
</button>
</form>
)}
</section>
);

```

4. Delete Record

The screenshot shows a web browser window with the URL `localhost:5173/delete`. The page title is "Community Sports League Fixtures". The user is signed in as "ananya (ananya@gmail.com)" and can click "Log out". The navigation bar includes links for "Home", "Create", "Update", and "Delete". The main content area is titled "Delete Fixture" and contains a form with a "Select fixture" dropdown menu showing "Round 2 — Sharks vs Falcons". Below the dropdown is a "Delete fixture" button.





Fixtures

Semi-final — sharks vs dolphins

Round 3 — Falcons vs Dolphins

Code:

```
export default function DeleteRecord({ user }) {
  const navigate = useNavigate();

  const [fixtures, setFixtures] = useState([]);
  const [selectedFixtureId, setSelectedFixtureId] = useState("");
  const [error, setError] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);

  useEffect(() => {
    if (!user) {
      return;
    }

    async function loadFixtures() {
      setIsLoading(true);
      setError("");

      try {
        const response = await fetch("/api/fixtures", {
          credentials: "include",
        });

        const data = await response.json();

        if (!response.ok) {
          throw new Error(data.detail || "Could not load fixtures");
        }

        setFixtures(data);
      } catch (requestError) {
        setError(requestError.message);
      } finally {
        setIsLoading(false);
      }
    }

    loadFixtures();
  }, [user]);
```

```
async function handleSubmit(event) {
  event.preventDefault();

  if (!selectedFixtureId) {
    setError("Select a fixture to delete.");
    return;
  }

  // loads the authenticated user's available fixtures
  const selectedFixture = fixtures.find(
    (fixture) => fixture.id === Number(selectedFixtureId),
  );

  // asks for explicit confirmation
  const shouldDelete = window.confirm(
    `Delete "${selectedFixture.fixture_name}"?`,
  );

  if (!shouldDelete) {
    return;
  }

  setError("");
  setIsSubmitting(true);

  // Sends a DELETE request to the backend
  try {
    const response = await fetch(
      `/api/fixtures/${selectedFixtureId}`,
      {
        method: "DELETE",
        credentials: "include",
      },
    );

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.detail || "Could not delete fixture");
    }

    navigate("/");
  } catch (error) {
    setError(error.message);
  }
}
```

```

    } catch (requestError) {
      setError(requestError.message);
    } finally {
      setIsSubmitting(false);
    }
  }

  if (!user) {
    return (
      <section>
        <h2>Delete Fixture</h2>
        <p>Please log in before deleting a fixture.</p>
      </section>
    );
  }

  return (
    <section>
      <h2>Delete Fixture</h2>

      {isLoading && <p>Loading fixtures...</p>}

      {!isLoading && (
        <form onSubmit={handleSubmit}>
          <label>
            Select fixture
            <select
              value={selectedFixtureId}
              onChange={(event) => setSelectedFixtureId(event.target.value)}
              required
            >
              <option value="">Choose a fixture</option>

              {fixtures.map((fixture) => (
                <option key={fixture.id} value={fixture.id}>
                  {fixture.fixture_name} – {fixture.teams}
                </option>
              ))}
            </select>
          </label>
        </form>
      )}
    </section>
  );
}

```

```

      {error && <p role="alert">{error}</p>}

      <button type="submit" disabled={isSubmitting}>
        {isSubmitting ? "Deleting..." : "Delete fixture"}
      </button>
    </form>
  )}
</section>
);
}

```

5. React Router - (other components like props and hooks are also in the previous code screenshots)

main.jsx

```
code > web_application > frontend > src > main.jsx
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import { BrowserRouter } from "react-router-dom";
4
5  import App from "./App.jsx";
6  import "./index.css";
7
8  createRoot(document.getElementById("root")).render(
9    <StrictMode>
10     <BrowserRouter>
11       <App />
12     </BrowserRouter>
13   </StrictMode>,
14 );
```

Auth.jsx

```
export default function App() {
  const [user, setUser] = useState(null);
  const [isCheckingSession, setIsCheckingSession] = useState(true);

  useEffect(() => { // runs once when app first loads
    async function checkSession() {
      try {
        const response = await fetch("/api/me", { // checks whether the browser
          credentials: "include",
        });

        if (response.ok) { // If valid, it restores the signed-in user into React
          const currentUser = await response.json();
          setUser(currentUser);
        }
      } finally {
        setIsCheckingSession(false);
      }
    }

    checkSession();
  }, []);

  async function handleLogout() { // removes the database session and cookie
    await fetch("/api/logout", {
      method: "POST",
      credentials: "include",
    });

    setUser(null);
  }
}
```

```

return (
  <>
    <header className="site-header">
      <div className="header-content">
        <div className="header-top">
          <h1>Community Sports League Fixtures</h1>

          {isCheckingSession ? (
            <p className="session-status">Checking session...</p>
          ) : user ? (
            <div className="account-actions">
              <span>
                Signed in as {user.name} ({user.email})
              </span>
              <button type="button" onClick={handleLogout}>
                Log out
              </button>
            </div>
          ) : (
            <p className="session-status">Not signed in</p>
          )}
        </div>

        <nav aria-label="Primary navigation">
          <NavLink to="/">Home</NavLink>
          <NavLink to="/create">Create</NavLink>
          <NavLink to="/update">Update</NavLink>
          <NavLink to="/delete">Delete</NavLink>
          {!user && !isCheckingSession && (
            <NavLink to="/login">Login</NavLink>
          )}
        </nav>
      </div>
    </header>

    <main className="page-content">
      <Routes>
        <Route path="/" element={<Home user={user} />} />
        <Route path="/login" element={<Login onLogin={setUser} />} />
        <Route path="/create" element={<CreateRecord user={user} />} />
        <Route path="/update" element={<UpdateRecord user={user} />} />
        <Route path="/delete" element={<DeleteRecord user={user} />} />
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </main>
  </>
)

```

Part - 2 MySQL Persistence and Server-Side Sessions

1. Login - Post

The screenshot shows a REST client interface for a 'HW4 Fixture API'. A new request is being created with the following details:

- Method:** POST
- URL:** `{{base_url}} /api/login`
- Body:** A JSON object:

```
{  "email": "ananya@gmail.com",  "password": "homework"}
```

The request is sent, resulting in a **200 OK** response. The response body is displayed in JSON format:

```
{  "message": "Login successful",  "name": "ananya",  "email": "ananya@gmail.com"}
```

The response status is 200 OK, with a response time of 82 ms and a size of 317 B.

2. Home - Get (Protected)

The screenshot shows a REST client interface for the same 'HW4 Fixture API'. A new request is being created with the following details:

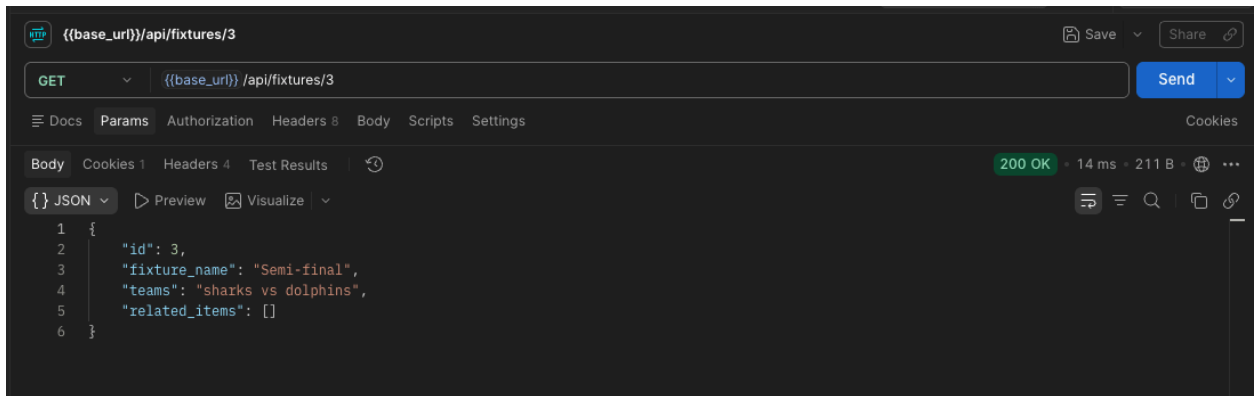
- Method:** GET
- URL:** `{{base_url}} /api/fixtures`

The request is sent, resulting in a **200 OK** response. The response body is displayed in JSON format:

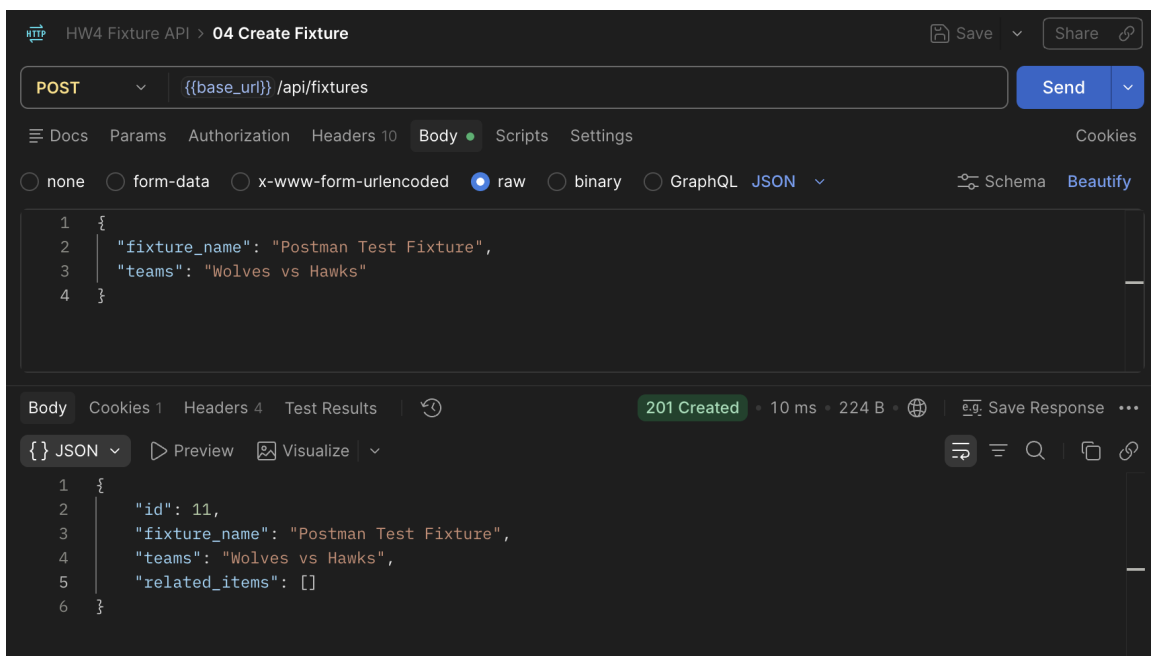
```
[  {    "id": 3,    "fixture_name": "Semi-final",    "teams": "sharks vs dolphins",    "related_items": []  },  {    "id": 6,    "fixture_name": "Round 3",    "teams": "Falcons vs Dolphins",    "related_items": []  }]
```

The response status is 200 OK, with a response time of 15 ms and a size of 297 B.

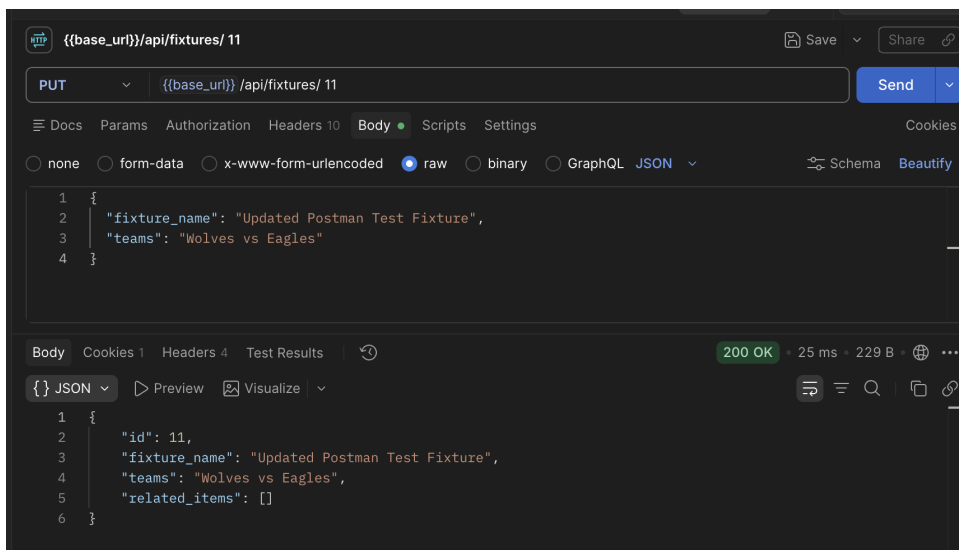
3. View record by id - Get



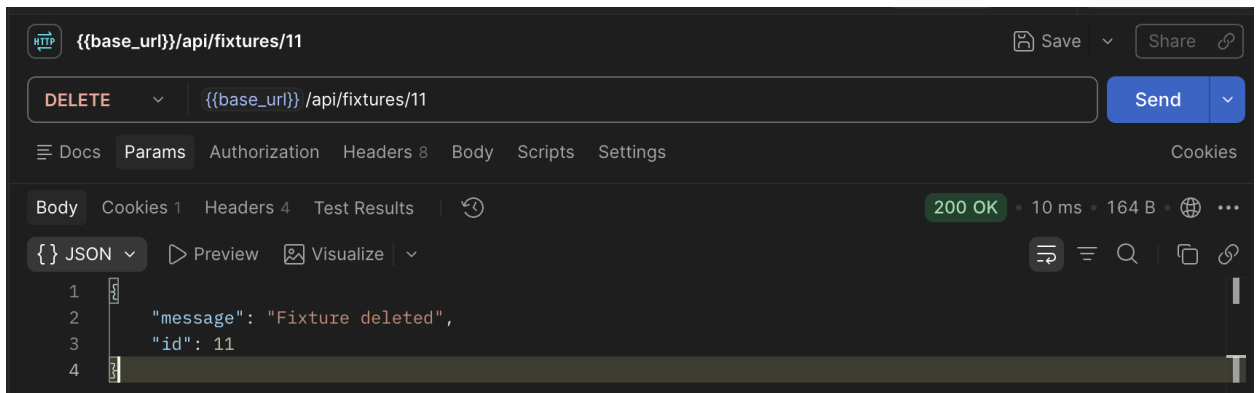
4. Add a new record - Post



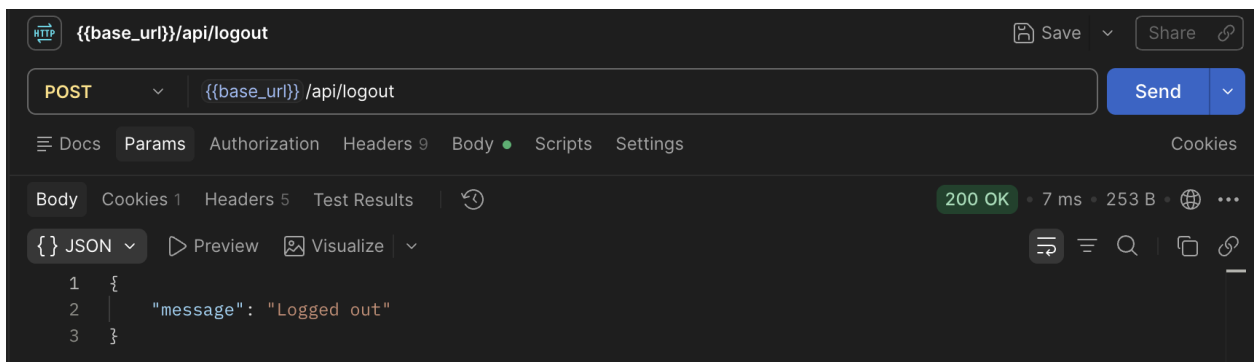
5. Update Record - Put



6. Delete a record - Delete



7. Logout - Post



Database and session evidence:

```
mysql> USE s2383_rel;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_s2383_rel |
+-----+
| fixture_related_items |
| fixtures              |
| sessions              |
| users                 |
+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT id, user_id, expires_at FROM sessions;
+-----+-----+-----+
| id                                     | user_id | expires_at |
+-----+-----+-----+
| DycP7M5BgxYh_LxIqeG9vplGHg90Jagcl0bL4rBv3v8 | 2       | 2026-09-27 22:12:14 |
| Il0_8PSi8feU7gV6Nlc_mRbTxIBf01PLUPRDGySkrFw | 2       | 2026-09-27 19:36:35 |
| qylRUtHjoNy2ZhtXpDr7kK__11wVM0jK9_5CPZQk90E | 2       | 2026-09-28 05:50:57 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Code:

db.py

```
5 connection_values = {
6     "drivername": "mysql+pymysql",
7     "username": os.environ["MYSQL_USER"],
8     "password": os.environ["MYSQL_PASSWORD"],
9     "database": os.environ["MYSQL_DATABASE"],
10 }
11
12
13 if not socket_path:
14     connection_values["host"] = os.environ["MYSQL_HOST"]
15     connection_values["port"] = int(os.environ["MYSQL_PORT"])
16
17 database_url = URL.create(**connection_values)
18
19 engine_options = {
20     "pool_pre_ping": True,
21 }
22
23 if socket_path:
24     engine_options["connect_args"] = {
25         "unix_socket": socket_path,
26     }
27
28 engine = create_engine(
29     database_url,
30     **engine_options,
31 )
32
33 SessionLocal = sessionmaker(
34     autocommit=False,
35     autoflush=False,
36     bind=engine,
37 )
38
39
40 class Base(DeclarativeBase):
41     pass
42
43
44 def get_db():
45     db = SessionLocal()
46
47     try:
48         yield db
49     finally:
50         db.close()
```

main.py

```
app = FastAPI(
    title="Community Sports League Fixtures",
)

app.include_router(auth_router)

@app.on_event("startup")
def create_tables():
    Base.metadata.create_all(bind=engine)

class FixtureCreate(BaseModel):
    fixture_name: str
    teams: str

class FixtureUpdate(BaseModel):
    fixture_name: str
    teams: str

def fixture_to_dict(fixture: Fixture):
    return {
        "id": fixture.id,
        "fixture_name": fixture.fixture_name,
        "teams": fixture.teams,
        "related_items": [
            {
                "id": item.id,
                "label": item.label,
                "details": item.details,
            }
            for item in fixture.related_items
        ],
    }
```

```
def root():
    return {
        "message": "Community Sports League Fixtures API",
    }

@app.get("/api/fixtures")
def list_fixtures(
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    fixtures = (
        db.query(Fixture)
        .options(selectinload(Fixture.related_items))
        .order_by(Fixture.id)
        .all()
    )

    return [
        fixture_to_dict(fixture)
        for fixture in fixtures
    ]

@app.get("/api/fixtures/{fixture_id}")
def get_fixture(
    fixture_id: int,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    fixture = (
        db.query(Fixture)
        .options(selectinload(Fixture.related_items))
        .filter(Fixture.id == fixture_id)
        .first()
    )

    if not fixture:
        raise HTTPException(
            status_code=404,
            detail="Fixture not found",
        )

    return fixture_to_dict(fixture)
```

```

@app.post("/api/fixtures", status_code=201)
def create_fixture(
    request_data: FixtureCreate,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    fixture = Fixture(
        fixture_name=request_data.fixture_name,
        teams=request_data.teams,
    )

    db.add(fixture)
    db.commit()
    db.refresh(fixture)

    return fixture_to_dict(fixture)


@app.put("/api/fixtures/{fixture_id}")
def update_fixture(
    fixture_id: int,
    request_data: FixtureUpdate,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    fixture = db.get(Fixture, fixture_id)

    if not fixture:
        raise HTTPException(
            status_code=404,
            detail="Fixture not found",
        )

    fixture.fixture_name = request_data.fixture_name
    fixture.teams = request_data.teams

    db.commit()
    db.refresh(fixture)

    return fixture_to_dict(fixture)

```

```

@app.delete("/api/fixtures/{fixture_id}")
def delete_fixture(
    fixture_id: int,
    db: DbSession = Depends(get_db),
    user: User = Depends(get_current_user),
):
    fixture = db.get(Fixture, fixture_id)

    if not fixture:
        raise HTTPException(
            status_code=404,
            detail="Fixture not found",
        )

    db.delete(fixture)
    db.commit()

    return {
        "message": "Fixture deleted",
        "id": fixture_id,
    }


if __name__ == "__main__":
    uvicorn.run(
        app,
        port=int(os.environ["APP_PORT"]),
    )

```

auth.py

```
SESSION_COOKIE_NAME = "s2383_session"
SESSION_DURATION = timedelta(hours=1)

class LoginRequest(BaseModel):
    email: str
    password: str

class RegisterRequest(BaseModel):
    name: str
    email: str
    password: str

def get_current_user(
    request: Request,
    db: DbSession = Depends(get_db),
) -> User:
    session_token = request.cookies.get(SESSION_COOKIE_NAME)

    if not session_token:
        raise HTTPException(
            status_code=401,
            detail="Login required",
        )

    session_record = db.get(Session, session_token)

    if not session_record:
        raise HTTPException(
            status_code=401,
            detail="Login required",
        )

    if session_record.expires_at <= datetime.utcnow():
        db.delete(session_record)
        db.commit()

        raise HTTPException(
            status_code=401,
            detail="Login required",
        )

    return session_record.user
```

```
@router.post("/api/register")
def register(
    request_data: RegisterRequest,
    db: DbSession = Depends(get_db),
):
    existing_user = (
        db.query(User)
        .filter(User.email == request_data.email)
        .first()
    )

    if existing_user:
        raise HTTPException(
            status_code=409,
            detail="Email is already registered",
        )

    user = User(
        name=request_data.name,
        email=request_data.email,
        password_hash=password_hash.hash(
            request_data.password
        ),
    )

    db.add(user)
    db.commit()
    db.refresh(user)

    return {
        "id": user.id,
        "name": user.name,
        "email": user.email,
    }
```

```

@router.post("/api/login")
def login(
    request_data: LoginRequest,
    response: Response,
    db: DbSession = Depends(get_db),
):
    user = (
        db.query(User)
        .filter(User.email == request_data.email)
        .first()
    )

    if not user or not password_hash.verify(
        request_data.password,
        user.password_hash,
    ):
        raise HTTPException(
            status_code=401,
            detail="Invalid email or password",
        )

    session_token = secrets.token_urlsafe(32)

    session_record = Session(
        id=session_token,
        user_id=user.id,
        expires_at=datetime.utcnow() + SESSION_DURATION,
    )

    db.add(session_record)
    db.commit()

    response.set_cookie(
        key=SESSION_COOKIE_NAME,
        value=session_token,
        httponly=True,
        samesite="lax",
        secure=False,
        max_age=int(SESSION_DURATION.total_seconds()),
    )

    return {
        "message": "Login successful",
        "name": user.name,
        "email": user.email,
    }

```

```

@router.post("/api/logout")
def logout(
    request: Request,
    response: Response,
    db: DbSession = Depends(get_db),
):
    session_token = request.cookies.get(SESSION_COOKIE_NAME)

    if session_token:
        session_record = db.get(Session, session_token)

        if session_record:
            db.delete(session_record)
            db.commit()

    response.delete_cookie(SESSION_COOKIE_NAME)

    return {"message": "Logged out"}

@router.get("/api/me")
def current_user(
    user: User = Depends(get_current_user),
):
    return {
        "id": user.id,
        "name": user.name,
        "email": user.email,
    }

```

Part 3 - N+1 Measurement and Query Tuning

1. Generating records

```
mysql> SELECT COUNT(*) AS seeded_fixture_count
-> FROM fixtures
-> WHERE fixture_name LIKE 'HW4 Fixture %';
+-----+
| seeded_fixture_count |
+-----+
|                    5000 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT COUNT(*) AS seeded_related_item_count
-> FROM fixture_related_items AS related_item
-> JOIN fixtures AS fixture
-> ON fixture.id = related_item.fixture_id
-> WHERE fixture.fixture_name LIKE 'HW4 Fixture %';
+-----+
| seeded_related_item_count |
+-----+
|                    200 |
+-----+
1 row in set (0.00 sec)
```

Naive Fixtures - 10

HW4 Fixture API - 08 Naive Fixtures - 10

GET {{base_url}} /api/fixtures/naive?page_size=10

200 OK • 12 ms • 1.09 KB

JSON

```
1 {
2   {
3     "id": 3,
4     "fixture_name": "Semi-final",
5     "teams": "sharks vs dolphins",
6     "related_items": []
7   },
8   {
9     "id": 6,
10    "fixture_name": "Round 3",
11    "teams": "Falcons vs Dolphins",
12    "related_items": []
13  },
14  {
15    "id": 12,
16    "fixture_name": "HW4 Fixture 0001",
17    "teams": "Team 54 vs Team 21",
18    "related_items": []
19  },
20  {
21    "id": 13,
22    "fixture_name": "HW4 Fixture 0002",
23    "teams": "Team 1 vs Team 58",
24    "related_items": []
25  },
26  {
27    "id": 14,
28    "fixture_name": "HW4 Fixture 0003",
29    "teams": "Team 16 vs Team 67",
30    "related_items": []
31  },
32  {
33    "id": 15,
34    "fixture_name": "HW4 Fixture 0004",
35    "teams": "Team 32 vs Team 38",
36    "related_items": []
37  },
```

```
},
{
  "id": 16,
  "fixture_name": "HW4 Fixture 0005",
  "teams": "Team 33 vs Team 89",
  "related_items": []
},
{
  "id": 17,
  "fixture_name": "HW4 Fixture 0006",
  "teams": "Team 42 vs Team 39",
  "related_items": []
},
{
  "id": 18,
  "fixture_name": "HW4 Fixture 0007",
  "teams": "Team 99 vs Team 69",
  "related_items": [
    {
      "id": 81,
      "label": "Match note",
      "details": "Generated related item for HW4 Fixture 0007."
    }
  ]
},
{
  "id": 19,
  "fixture_name": "HW4 Fixture 0008",
  "teams": "Team 77 vs Team 35",
  "related_items": []
}
```

Fixed Fixtures - 10

The screenshot shows a REST client interface for the 'HW4 Fixture API'. The endpoint is `GET {{base_url}} /api/fixtures?page_size=10`. The response is a JSON array of 10 fixture objects, each with an id, fixture_name, teams, and related_items. The status is 200 OK.

```
1 [
2   {
3     "id": 3,
4     "fixture_name": "Semi-final",
5     "teams": "sharks vs dolphins",
6     "related_items": []
7   },
8   {
9     "id": 6,
10    "fixture_name": "Round 3",
11    "teams": "Falcons vs Dolphins",
12    "related_items": []
13  },
14  {
15    "id": 12,
16    "fixture_name": "HW4 Fixture 0001",
17    "teams": "Team 54 vs Team 21",
18    "related_items": []
19  },
20  {
21    "id": 13,
22    "fixture_name": "HW4 Fixture 0002",
23    "teams": "Team 1 vs Team 58",
24    "related_items": []
25  },
26  {
27    "id": 14,
28    "fixture_name": "HW4 Fixture 0003",
29    "teams": "Team 16 vs Team 67",
30    "related_items": []
31  },
32  {
33    "id": 15,
34    "fixture_name": "HW4 Fixture 0004",
35    "teams": "Team 32 vs Team 38",
36    "related_items": []
37  },
38  {
39    "id": 16,
40    "fixture_name": "HW4 Fixture 0005",
41    "teams": "Team 33 vs Team 89",
42    "related_items": []
43  },
44  {
45    "id": 17,
46    "fixture_name": "HW4 Fixture 0006",
47    "teams": "Team 42 vs Team 39",
48    "related_items": []
49  },
50  {
51    "id": 18,
52    "fixture_name": "HW4 Fixture 0007",
53    "teams": "Team 99 vs Team 69",
54    "related_items": [
55      {
56        "id": 81,
57        "label": "Match note",
58        "details": "Generated related item for HW4 Fixture 0007."
59      }
60    ]
61  },
62  {
63    "id": 19,
64    "fixture_name": "HW4 Fixture 0008",
65    "teams": "Team 77 vs Team 35",
66    "related_items": []
67  }
68 ]
```

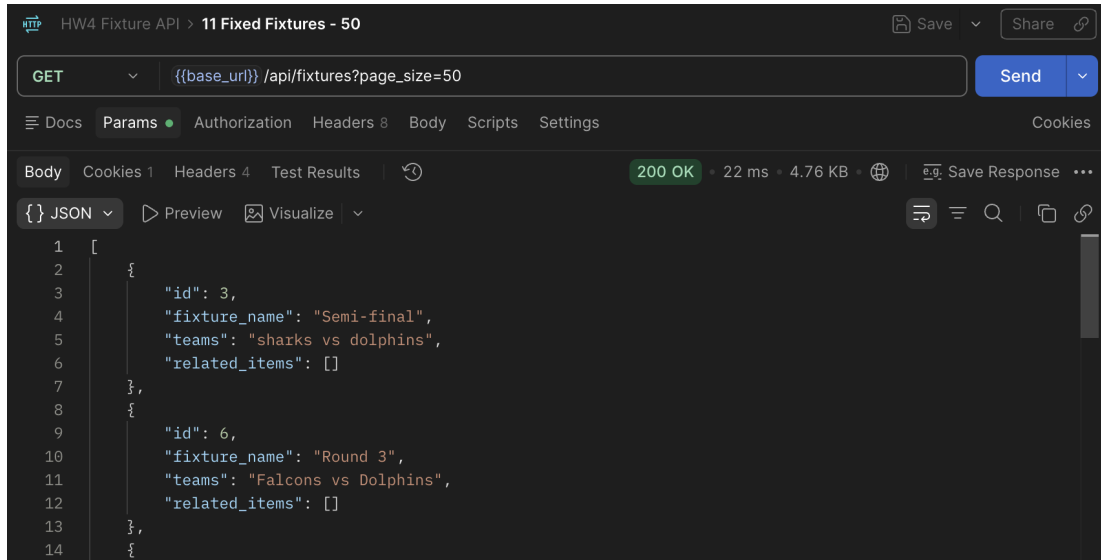
```
{
  "id": 16,
  "fixture_name": "HW4 Fixture 0005",
  "teams": "Team 33 vs Team 89",
  "related_items": []
},
{
  "id": 17,
  "fixture_name": "HW4 Fixture 0006",
  "teams": "Team 42 vs Team 39",
  "related_items": []
},
{
  "id": 18,
  "fixture_name": "HW4 Fixture 0007",
  "teams": "Team 99 vs Team 69",
  "related_items": [
    {
      "id": 81,
      "label": "Match note",
      "details": "Generated related item for HW4 Fixture 0007."
    }
  ]
},
{
  "id": 19,
  "fixture_name": "HW4 Fixture 0008",
  "teams": "Team 77 vs Team 35",
  "related_items": []
}
```

Naive Fixtures - 50 (not showing all 50 records with interest of space)

The screenshot shows a REST client interface for the 'HW4 Fixture API'. The endpoint is `GET {{base_url}} /api/fixtures/naive?page_size=50`. The response is a JSON array of 50 fixture objects. The status is 200 OK.

```
1 [
2   {
3     "id": 3,
4     "fixture_name": "Semi-final",
5     "teams": "sharks vs dolphins",
6     "related_items": []
7   },
8   {
9     "id": 6,
10    "fixture_name": "Round 3",
11    "teams": "Falcons vs Dolphins",
12    "related_items": []
13  },
14  {
15    "id": 12,
16    "fixture_name": "HW4 Fixture 0001",
17    "teams": "Team 54 vs Team 21",
18    "related_items": []
19  },
20  {
21    "id": 13,
```

Fixed Fixtures - 50



HW4 Fixture API > 11 Fixed Fixtures - 50

GET {{base_url}} /api/fixtures?page_size=50

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body Cookies 1 Headers 4 Test Results

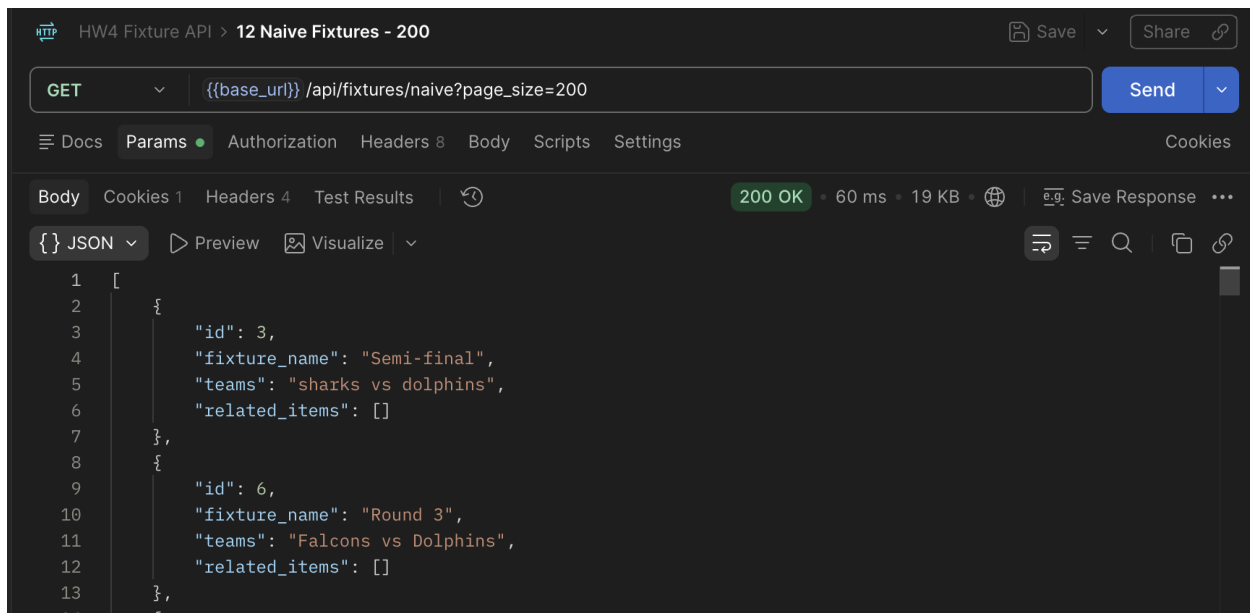
200 OK • 22 ms • 4.76 KB

Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 3,
4     "fixture_name": "Semi-final",
5     "teams": "sharks vs dolphins",
6     "related_items": []
7   },
8   {
9     "id": 6,
10    "fixture_name": "Round 3",
11    "teams": "Falcons vs Dolphins",
12    "related_items": []
13  },
14  ]
```

Naive Fixtures - 200



HW4 Fixture API > 12 Naive Fixtures - 200

GET {{base_url}} /api/fixtures/naive?page_size=200

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body Cookies 1 Headers 4 Test Results

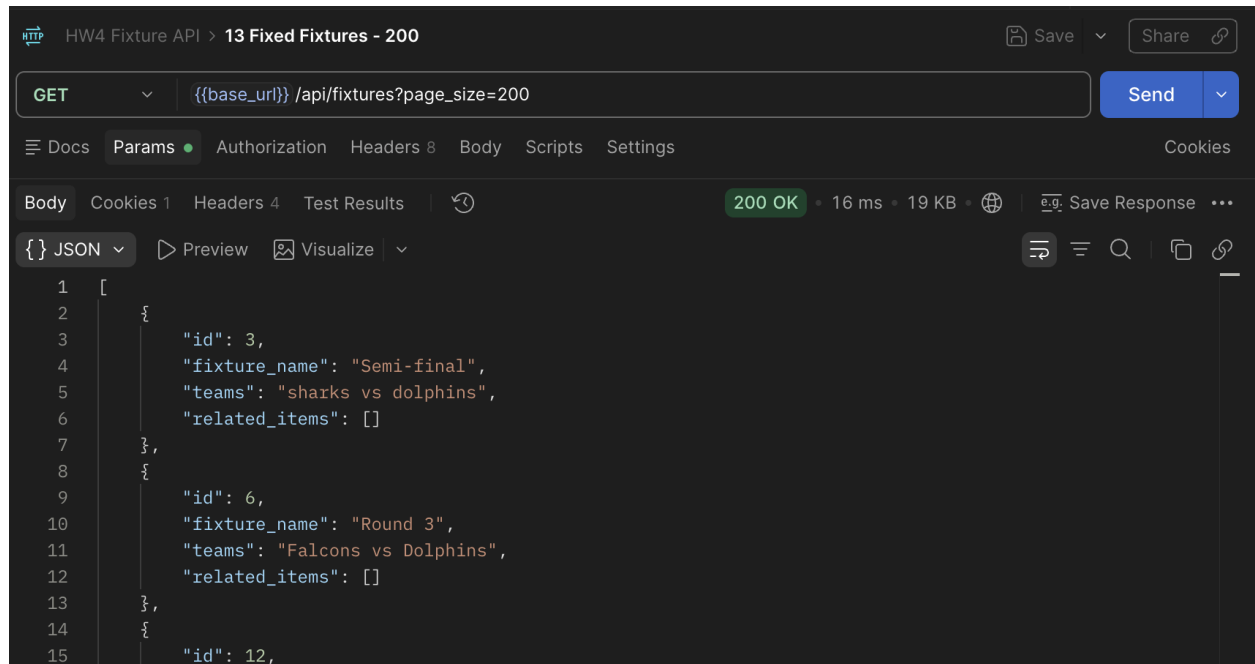
200 OK • 60 ms • 19 KB

Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 3,
4     "fixture_name": "Semi-final",
5     "teams": "sharks vs dolphins",
6     "related_items": []
7   },
8   {
9     "id": 6,
10    "fixture_name": "Round 3",
11    "teams": "Falcons vs Dolphins",
12    "related_items": []
13  },
14  ]
```

Fixed Fixtures - 200



HW4 Fixture API > 13 Fixed Fixtures - 200

GET {{base_url}} /api/fixtures?page_size=200

Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Body Cookies 1 Headers 4 Test Results 200 OK • 16 ms • 19 KB • Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": 3,
4     "fixture_name": "Semi-final",
5     "teams": "sharks vs dolphins",
6     "related_items": []
7   },
8   {
9     "id": 6,
10    "fixture_name": "Round 3",
11    "teams": "Falcons vs Dolphins",
12    "related_items": []
13  },
14  {
15    "id": 12,
```

Code:

```
55
56 @app.get("/api/fixtures")
57 def list_fixtures(
58     page_size: int = Query(default=10, ge=1, le=200),
59     db: DbSession = Depends(get_db),
60     user: User = Depends(get_current_user),
61 ):
62     fixtures = (
63         db.query(Fixture)
64         .options(selectinload(Fixture.related_items))
65         .order_by(Fixture.id)
66         .limit(page_size)
67         .all()
68     )
69
70     return [
71         fixture_to_dict(fixture)
72         for fixture in fixtures
73     ]
74
75 @app.get("/api/fixtures/naive")
76 def list_fixtures_naive( # Without selectinload, SQLAlchemy w
77     page_size: int = Query(default=10, ge=1, le=200),
78     db: DbSession = Depends(get_db),
79     user: User = Depends(get_current_user),
80 ):
81     fixtures = (
82         db.query(Fixture)
83         .order_by(Fixture.id)
84         .limit(page_size)
85         .all()
86     )
87
88     return [
89         fixture_to_dict(fixture)
90         for fixture in fixtures
91     ]
92
```

Extra query count shown for naive (10):

Body	Cookies 1	Headers 5	Test Results	🔄	200 OK	• 18 ms • 1.12 KB • 🌐	🔗 Save Response	...
Key		Value						
Date		Mon, 28 Sep 2026 07:58:35 GMT						
Server		uvicorn						
X-Sql-Query-Count		11						
Content-Length		991						
Content-Type		application/json						

Fixed does not have extra (10):

Body	Cookies 1	Headers 5	Test Results	🔄	200 OK	• 10 ms • 1.11 KB • 🌐	🔗 Save Response	...
Key		Value						
Date		Mon, 28 Sep 2026 07:59:22 GMT						
Server		uvicorn						
X-Sql-Query-Count		2						
Content-Length		991						
Content-Type		application/json						

2. Index improves a filtered lookup. Eager loading fixes excessive related-item queries:

```
mysql> EXPLAIN
-> SELECT id, fixture_name, teams
-> FROM fixtures
-> WHERE fixture_name LIKE 'HM4 Fixture 2%';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	fixtures	NULL	ALL	NULL	NULL	NULL	NULL	4982	11.11	Using where

```
1 row in set, 1 warning (0.00 sec)
```

```
mysql> CREATE INDEX ix_fixtures_fixture_name
-> ON fixtures (fixture_name);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> EXPLAIN
-> SELECT id, fixture_name, teams
-> FROM fixtures
-> WHERE fixture_name LIKE 'HM4 Fixture 2%';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	fixtures	NULL	range	ix_fixtures_fixture_name	ix_fixtures_fixture_name	1022	NULL	1000	100.00	Using index condition

```
1 row in set, 1 warning (0.00 sec)
```

Naive 50 -

Key	Value
Date	Mon, 28 Sep 2026 08:58:52 GMT
Server	uvicorn
X-Sql-Query-Count	51
Content-Length	4749
Content-Type	application/json

Fixed 50 -

Body	Cookies 1	Headers 5	Test Results		200 OK • 7 ms • 4.79 KB • 🌐
Key	Value				
Date	Mon, 28 Sep 2026 08:59:23 GMT				
Server	uvicorn				
X-Sql-Query-Count	2				
Content-Length	4749				
Content-Type	application/json				

Naive 200 -

BodyCookies 1Headers 5Test Results200 OK59 ms19.03 KB

Key	Value
Date	Mon, 28 Sep 2026 08:59:54 GMT
Server	uvicorn
X-Sql-Query-Count	201
Content-Length	19330
Content-Type	application/json

Fixed 200 -

Body	Cookies 1	Headers 5	Test Results		200 OK • 12 ms • 19.03 KB •
Key	Value				
Date	Mon, 28 Sep 2026 09:00:14 GMT				
Server	uvicorn				
X-Sql-Query-Count	2				
Content-Length	19330				
Content-Type	application/json				

Naïve versus fixed endpoint performance across page sizes

Percentiles use the nearest-rank method.

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naïve	11	3.305	3.863	4.603
10	fixed	2	1.453	1.670	2.157
50	naïve	51	8.398	11.114	11.377
50	fixed	2	1.838	2.139	2.360
200	naïve	201	28.919	34.089	38.089
200	fixed	2	3.107	3.517	17.197

Fixed Endpoint Latency Improvement

The fixed endpoint reduced p50 latency at every page size. At page size 10, p50 decreased from 3.305 ms to 1.453 ms. At page size 50, it decreased from 8.398 ms to 1.838 ms. At page size 200, it decreased from 28.919 ms to 3.107 ms.

The improvement became larger as page size increased because the naïve endpoint performed one related-item query for every fixture returned. Its SQL statement count therefore grew from 11 to 51 and then 201 (basically $N+1$). The fixed endpoint used eager loading and remained at two SQL statements for every page size, one query for fixtures and one query for their related items.

Index EXPLAIN Comparison

Before adding the index, MySQL scanned the full fixtures table for the fixture_name LIKE 'HW4 Fixture 2%' query. The EXPLAIN output showed type = ALL, no selected index, and an estimate of 4,982 rows examined.

After adding the fixture_name index, MySQL used the ix_fixtures_fixture_name index with type = range. The estimated rows examined decreased to 1,000, and the output showed Using index condition. This shows that MySQL could use the index for the prefix search instead of scanning the full table.

Index snippet

```
2
3 class Fixture(Base):
4     __tablename__ = "fixtures"
5
6     id: Mapped[int] = mapped_column(
7         Integer,
8         primary_key=True,
9         autoincrement=True,
10    )
11    fixture_name: Mapped[str] = mapped_column(
12        String(255),
13        nullable=False,
14        index=True,
15    )
16    teams: Mapped[str] = mapped_column(
17        String(255),
18        nullable=False,
19    )
20
21    related_items: Mapped[list["FixtureRelatedItem"]] = relationship(
22        back_populates="fixture",
23        cascade="all, delete-orphan",
24    )
25
26
```

Part 4 - Build a Grounded RAG Question-Answering System

1. Run k sweep output json file (for only q2):

```
{
  "question": {
    "id": "q2",
    "type": "two_chunks",
    "question": "In a FastAPI application, how are request bodies validated and how is the application started with Uvicorn?",
    "expected_refusal": false
  },
  "configuration": "context_rag",
  "results": [
    {
      "k": 1,
      "retrieved_chunks": [
        {
          "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
          "page": 47,
          "chunk": 1,
          "score": 0.7798317670822144,
          "text": "FastAPI is the web framework. Uvicorn is the ASGI server that runs it. It is ASGI server implementation"
        }
      ],
      "answer": "In a FastAPI application, request bodies are validated using Pydantic models, which are integrated into FastAPI for data validation. The application is started with Uvicorn by running the FastAPI application as the ASGI application, typically using the command `uvicorn main:app --reload`, where `main` is the module and `app` is the FastAPI instance. \n\nSource: [Source 1: 01-Rest_FastAPI_Agent_v3.pptx.pdf, page 47]"
    },
    {
      "k": 3,
      "retrieved_chunks": [
        {
          "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
          "page": 47,
          "chunk": 1,
          "score": 0.7798317670822144,
          "text": "FastAPI is the web framework. Uvicorn is the ASGI server that runs it. It is ASGI server implementation"
        }
      ],

```

```

{
  "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
  "page": 5,
  "chunk": 1,
  "score": 0.7236416935920715,
  "text": "Web APIs: HTTP \u2192 REST \u2192 FastAPI Goal: design
RESTful resources and implement them cleanly in FastAPI We\u2019ll connect:
methods \u2192 URIs \u2192 representations \u2192 status codes \u2192
validation \u2192 auth"
},
{
  "source": "02-Session FastAPI.LLM Intro .pptx.pdf",
  "page": 7,
  "chunk": 2,
  "score": 0.7126616835594177,
  "text": "e web framework pieces (routing, middleware, \u25cf
requests/responses, WebSockets). \u25cf FastAPI adds Pydantic-based validation
and automatic OpenAPI docs."
}
],
"answer": "In a FastAPI application, request bodies are validated using
Pydantic-based validation, which is a feature provided by FastAPI. This is
mentioned in [Source 3]. The application is started with Uvicorn as the ASGI
server, which is noted in [Source 1]. \n\nCitations: [Source 1], [Source 3]"
},
{
  "k": 5,
  "retrieved_chunks": [
    {
      "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
      "page": 47,
      "chunk": 1,
      "score": 0.7798317670822144,
      "text": "FastAPI is the web framework. Uvicorn is the ASGI server that
runs it. It is ASGI server implementation"
    },
    {
      "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
      "page": 5,
      "chunk": 1,
      "score": 0.7236416935920715,

```

```

        "text": "Web APIs: HTTP \u2192 REST \u2192 FastAPI Goal: design RESTful resources and implement them cleanly in FastAPI We\u2019ll connect: methods \u2192 URIs \u2192 representations \u2192 status codes \u2192 validation \u2192 auth"
    },
    {
        "source": "02-Session FastAPI.LLM Intro .pptx.pdf",
        "page": 7,
        "chunk": 2,
        "score": 0.7126616835594177,
        "text": "e web framework pieces (routing, middleware, \u25cf requests/responses, WebSockets). \u25cf FastAPI adds Pydantic-based validation and automatic OpenAPI docs."
    },
    {
        "source": "02-Session FastAPI.LLM Intro .pptx.pdf",
        "page": 14,
        "chunk": 1,
        "score": 0.710213840007782,
        "text": "Installing FastAPI 1. Create a directory myapp: mkdir myapp; cd myapp 2. Create a virtual environment: python -m venv .venv 3. Activate virtual environment: macOS/Linux: source .venv/bin/activate Windows: .venv\\Scripts\\activate 4. Install FastAPI + ASGI server: pip install fastapi uvicorn[standard] NOTE: Common entry point is main.py (similar idea to choosing app.js as entry point)."
    },
    {
        "source": "00-Introduction.pptx.pdf",
        "page": 17,
        "chunk": 1,
        "score": 0.6960365772247314,
        "text": "WEB Application Architecture (backend) FastAPI (Backend)"
    }
],
    "answer": "In a FastAPI application, request bodies are validated using Pydantic-based validation, which is a feature provided by FastAPI (as indicated in Source 3). \n\nTo start the application with Uvicorn, you typically install FastAPI and Uvicorn using pip, then run the application by executing the main.py file with Uvicorn as the ASGI server, as described in Source 4. \n\nCitations: Source 3, Source 4."
}
]

```

```
}
```

2. Rag comparison output json (for all six questions):

```
[
  {
    "id": "q1",
    "type": "one_chunk",
    "question": "What is a REST API?",
    "expected_refusal": false,
    "k": 3,
    "retrieved_chunks": [
      {
        "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
        "page": 16,
        "chunk": 1,
        "score": 0.8058532476425171,
        "text": "What is REST? \u2022 Representational State Transfer \u2022 Style of Software Architecture for WWW \u2022 Based on Client and Server \u2022 Request and Response are built on transfer of \u201crepresentation\u201d or \u201cresources\u201d \u2022 Uses HTTP protocol 16"
      },
      {
        "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
        "page": 5,
        "chunk": 1,
        "score": 0.7509481906890869,
        "text": "Web APIs: HTTP \u2192 REST \u2192 FastAPI Goal: design RESTful resources and implement them cleanly in FastAPI We\u2019ll connect: methods \u2192 URIs \u2192 representations \u2192 status codes \u2192 validation \u2192 auth"
      },
      {
        "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
        "page": 26,
        "chunk": 2,
        "score": 0.742737352848053,
        "text": "rom proper REST APIs? 26"
      }
    ]
  },
  {
    "answers": {
```

```

    "no_rag": "A REST API is an architectural style for building web services
using HTTP protocols, enabling communication between different systems through
stateless, client-server interactions.",
    "basic_rag": "A REST API is an application programming interface that uses
the principles of Representational State Transfer (REST) to enable
communication between different software systems. It is based on a
client-server architecture and utilizes HTTP protocols to request and respond
with representations of resources. REST APIs are designed to be stateless,
meaning each request from a client contains all the information the server
needs to fulfill that request. They are commonly used to build web services and
are often implemented using frameworks like FastAPI to design and implement
RESTful resources efficiently.",
    "context_rag": "A REST API is an application programming interface that
uses the principles of Representational State Transfer (REST) to enable
communication between different software systems. It is based on a
client-server architecture where requests and responses are built on the
transfer of representations or resources. REST APIs utilize the HTTP protocol
and are designed to interact with resources through standard HTTP methods such
as GET, POST, PUT, and DELETE. The design of REST APIs emphasizes stateless
interactions, uniform interfaces, and the ability to scale effectively. This
concept is further elaborated in the context of implementing RESTful resources
using FastAPI, which connects methods to URIs, representations, status codes,
validation, and authentication. (Source 1, Source 2)"
  }
},
{
  "id": "q2",
  "type": "two_chunks",
  "question": "In a FastAPI application, how are request bodies validated and
how is the application started with Uvicorn?",
  "expected_refusal": false,
  "k": 3,
  "retrieved_chunks": [
    {
      "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
      "page": 47,
      "chunk": 1,
      "score": 0.7798317670822144,
      "text": "FastAPI is the web framework. Uvicorn is the ASGI server that
runs it. It is ASGI server implementation"
    },
    {

```

```

    "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
    "page": 5,
    "chunk": 1,
    "score": 0.7236416935920715,
    "text": "Web APIs: HTTP \u2192 REST \u2192 FastAPI Goal: design RESTful
resources and implement them cleanly in FastAPI We\u2019ll connect: methods
\u2192 URIs \u2192 representations \u2192 status codes \u2192 validation \u2192
auth"
  },
  {
    "source": "02-Session FastAPI.LLM Intro .pptx.pdf",
    "page": 7,
    "chunk": 2,
    "score": 0.7126616835594177,
    "text": "e web framework pieces (routing, middleware, \u25cf
requests/responses, WebSockets). \u25cf FastAPI adds Pydantic-based validation
and automatic OpenAPI docs."
  }
],
"answers": {
  "no_rag": "In FastAPI, request bodies are validated using Pydantic models.
To start the application with Uvicorn, run `uvicorn main:app --reload` where
`main` is the module and `app` is the FastAPI instance.",
  "basic_rag": "In a FastAPI application, request bodies are validated using
Pydantic-based validation, which is a feature provided by FastAPI. This allows
for automatic data validation and documentation generation.\n\nTo start the
application with Uvicorn, you would typically run the FastAPI application using
Uvicorn as the ASGI server. This is done by executing a command such as
`uvicorn main:app --reload`, where `main` is the Python file containing the
FastAPI application and `app` is the FastAPI instance.",
  "context_rag": "In a FastAPI application, request bodies are validated
using Pydantic-based validation, which is a feature provided by FastAPI. This
is mentioned in [Source 3]. \n\nThe application is started with Uvicorn by
using Uvicorn as the ASGI server that runs the FastAPI web framework, as stated
in [Source 1]. \n\nCitations: [Source 1], [Source 3]"
}
},
{
  "id": "q3",
  "type": "similar_across_documents",
  "question": "What role does FastAPI play in a web application?",
  "expected_refusal": false,

```

```

"k": 3,
"retrieved_chunks": [
  {
    "source": "00-Introduction.pptx.pdf",
    "page": 17,
    "chunk": 1,
    "score": 0.8281787633895874,
    "text": "WEB Application Architecture (backend) FastAPI (Backend)"
  },
  {
    "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
    "page": 5,
    "chunk": 1,
    "score": 0.798352837562561,
    "text": "Web APIs: HTTP \u2192 REST \u2192 FastAPI Goal: design RESTful
resources and implement them cleanly in FastAPI We\u2019ll connect: methods
\u2192 URIs \u2192 representations \u2192 status codes \u2192 validation \u2192
auth"
  },
  {
    "source": "00-Introduction.pptx.pdf",
    "page": 19,
    "chunk": 1,
    "score": 0.7827216386795044,
    "text": "Stack + Kafka \u2022 MongoDB, FastAPI are dedicated to
developing the back-end of web applications. This corresponds to database
management, scripts, html documents, HTTP requests, etc. \u2022 React\u2019s
role is to execute HTTP requests. Fast API"
  }
],
"answers": {
  "no_rag": "FastAPI is a high-performance web framework for building APIs
with Python, offering features like automatic documentation, asynchronous
support, and type hints to streamline API development.",
  "basic_rag": "FastAPI plays the role of a backend framework in a web
application, responsible for handling HTTP requests, designing and implementing
RESTful resources, and managing communication between the client and the
database. It is used alongside technologies like MongoDB and Kafka to develop
the back-end of web applications, including database management, scripts, and
HTTP request handling.",
  "context_rag": "FastAPI plays the role of a backend framework in a web
application, dedicated to developing the back-end of web applications. This

```

includes tasks such as database management, scripts, HTML documents, and HTTP requests. It is used to design and implement RESTful resources cleanly, connecting methods to URIs, representations, status codes, validation, and authentication. \n\nSource numbers used: 1, 2, 3."

```
    }
  },
  {
    "id": "q4",
    "type": "ambiguous",
    "question": "What does the term model mean in the course materials?",
    "expected_refusal": false,
    "k": 3,
    "retrieved_chunks": [
      {
        "source": "03.ReactJS_MySQL_Prompt_RAG.pptx.pdf",
        "page": 39,
        "chunk": 1,
        "score": 0.6810703277587891,
        "text": "Model: Python class for a database table class Student(Base):
__tablename__ = \"students\" id = mapped_column(primary_key=True) name =
mapped_column(String(100)) major = mapped_column(String(100)) students table id
name major Model definitions are the \u201cmap\u201d between Python and table
structure. 7"
      },
      {
        "source": "03.ReactJS_MySQL_Prompt_RAG.pptx.pdf",
        "page": 58,
        "chunk": 1,
        "score": 0.6518493890762329,
        "text": "Context Engineering \u2014 Definition Context Engineering
designs everything the model sees at inference time, not just the prompt:
instructions, examples, retrieved documents, tools, memory/state, and control
\ufb02ow. \u25cf Prompt engineering = how you ask \u25cf Context engineering =
what you give the model to think with"
      },
      {
        "source": "02-Session FastAPI.LLM Intro .pptx.pdf",
        "page": 33,
        "chunk": 1,
        "score": 0.647320032119751,
        "text": "What is an LLM? \u25cf A probabilistic text model that predicts
the next token \u25cf Trained on large dataset makes it better at learning
```

```

statistical patterns & world knowledge \u25cf Transformer is the dominant
architecture \u25cf Typical capabilities: Q&A, summarization, extraction,
classi\ufb01cation, coding \u25cf Examples: GPT\u2011class models, Llama,
Gemma, Mistral, etc. 33"
    }
],
  "answers": {
    "no_rag": "In the course materials, the term \"model\" refers to a
simplified representation of a real-world system, process, or concept, used to
understand, analyze, or predict behavior.",
    "basic_rag": "The term \"model\" in the course materials refers to a
probabilistic text model that predicts the next token, trained on large
datasets to learn statistical patterns and world knowledge. It is also used to
describe a Python class for a database table, such as the `Student` class,
which maps Python code to a database table structure. Additionally, \"model\"
can relate to the broader concept of Context Engineering, where it encompasses
the design of everything the model sees during inference, including prompts,
examples, and other contextual information.",
    "context_rag": "In the course materials, the term \"model\" has at least
two distinct meanings, as supported by the provided documents:\n\n1. **A Python
class representing a database table**: In the context of database modeling, a
\"model\" refers to a Python class that maps to a database table structure. For
example, the `Student` class in the provided code snippet represents a table in
a database with columns such as `id`, `name`, and `major`. This usage of
\"model\" is related to object-relational mapping (ORM) and database design.
(Source 1)\n\n2. **A large language model (LLM)**: In the context of artificial
intelligence and natural language processing, \"model\" refers to a
probabilistic text model that predicts the next token based on a large dataset.
These models are trained using architectures like the Transformer and are
capable of tasks such as Q&A, summarization, and coding. Examples include
GPT-class models, Llama, Gemma, and Mistral. (Source 3)"
  }
},
{
  "id": "q5",
  "type": "not_in_documents",
  "question": "What is the maximum number of players allowed on a community
sports league soccer team?",
  "expected_refusal": true,
  "k": 3,
  "retrieved_chunks": [
    {

```

```

    "source": "03.ReactJS_MySQL_Prompt_RAG.pptx.pdf",
    "page": 72,
    "chunk": 2,
    "score": 0.5577385425567627,
    "text": "."
  },
  {
    "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
    "page": 74,
    "chunk": 3,
    "score": 0.5485806465148926,
    "text": "force TTL, size limits, and field schema. Guardrails sits
between your app and the LLM. 74"
  },
  {
    "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
    "page": 24,
    "chunk": 1,
    "score": 0.504791259765625,
    "text": "When is it appropriate? \u2022 The web services are completely
stateless \u2022 Caching infrastructure can be leveraged for performance \u2022
The service producer and consumer have mutual understanding of context and
content being passed e.g Netflix Api \u2022 Bandwidth is a constraint 24"
  }
],
"answers": {
  "no_rag": "The maximum number of players allowed on a community sports
league soccer team is typically 11.",
  "basic_rag": "The retrieved context does not provide information about the
maximum number of players allowed on a community sports league soccer team.",
  "context_rag": "I cannot answer this question from the provided
documents."
}
},
{
  "id": "q6",
  "type": "unrelated",
  "question": "What will the price of Bitcoin be on December 31, 2027?",
  "expected_refusal": true,
  "k": 3,
  "retrieved_chunks": [
    {

```

```

      "source": "01-Rest_FastAPI_Agent_v3.pptx.pdf",
      "page": 58,
      "chunk": 1,
      "score": 0.5539098381996155,
      "text": "Output Note: - Today's date in Asia/Kolkata is August 29, 2025
(Friday) at 02:13. - Today's date in America/Los_Angeles is August 28, 2025
(Thursday) at 13:43. Have a wonderful day filled with joy and laughter! 58"
    },
    {
      "source": "03.ReactJS_MySQL_Prompt_RAG.pptx.pdf",
      "page": 72,
      "chunk": 2,
      "score": 0.5317183136940002,
      "text": "."
    },
    {
      "source": "02-Session FastAPI.LLM Intro .pptx.pdf",
      "page": 53,
      "chunk": 2,
      "score": 0.5028182864189148,
      "text": "lower() for t in titles); print(count)\" \"\u25cf Observation:
987 \u25cf Final Answer: \u201c987 refund-related tickets last week; spike
followed a pricing change\u2014consider adding clearer refund terms on
checkout.\u201d"
    }
  ],
  "answers": {
    "no_rag": "The price of Bitcoin on December 31, 2027, cannot be predicted
with certainty, as it is influenced by many factors including market sentiment,
regulatory changes, technological developments, and macroeconomic conditions.",
    "basic_rag": "The provided context does not contain any information about
the price of Bitcoin on December 31, 2027. Therefore, I cannot answer the
question based on the given data.",
    "context_rag": "I cannot answer this question from the provided
documents."
  }
}
]

```

Evaluation Summary Table

Question	Configuration	Correct retrieval	Correct answer	Grounded	Refused when needed	Notes
Q1	No RAG	N/A	Yes	N/A	N/A	Correct general definition, but no document evidence was provided to the model.
Q1	Basic RAG	Yes	Yes	Partial	N/A	Correct answer, but it added "stateless" beyond the retrieved chunk text.
Q1	ContextRAG	Yes	Yes	Partial	N/A	Cited Sources 1 and 2, but added unsupported details such as specific HTTP methods.
Q2	No RAG	N/A	Yes	N/A	N/A	Correct general knowledge answer, with no supplied evidence.
Q2	Basic RAG	Yes	Yes	Partial	N/A	Retrieved both Pydantic and Uvicorn evidence, but added an unstated <code>uvicorn main:app --reload</code> command.
Q2	ContextRAG	Yes	Yes	Yes	N/A	Correctly combined Pydantic validation from Source 3 with Uvicorn from Source 1.

Q3	No RAG	N/A	Yes	N/A	N/A	Correct general answer, but no retrieved evidence was used.
Q3	Basic RAG	Yes	Yes	Yes	N/A	Correctly described FastAPI as a backend framework.
Q3	ContextRAG	Yes	Yes	Yes	N/A	Used consistent FastAPI evidence from two course documents.
Q4	No RAG	N/A	No	N/A	N/A	Interpreted "session" as a class meeting instead of the course's technical usage.
Q4	Basic RAG	Yes	Yes	Yes	N/A	Identified the database-model and LLM-model meanings.
Q4	ContextRAG	Yes	Yes	Yes	N/A	Clearly distinguished the two supported meanings with sources.
Q5	No RAG	N/A	No	N/A	No	Hallucinated a typical soccer-team size instead of refusing.
Q5	Basic RAG	Expected: no relevant evidence	Yes	Yes	Yes	Correctly said the context did not contain the answer.
Q5	ContextRAG	Expected: no relevant evidence	Yes	Yes	Yes	Used the exact required refusal sentence.

Q6	No RAG	N/A	No	N/A	No	Gave a general Bitcoin prediction disclaimer instead of a document-based refusal.
Q6	Basic RAG	Expected: no relevant evidence	Yes	Yes	Yes	Correctly identified that the context had no relevant information.
Q6	ContextRAG	Expected: no relevant evidence	Yes	Yes	Yes	Used the exact required refusal sentence.

Analysis: Retrieval and Context Engineering

I used five documents (the course lecture slides and the syllabus sheet) and split their text into 420 chunks with a chunk size of 500 characters and an overlap of 50 characters. For each question, the system retrieved the top three chunks and printed their source, page, and score before calling the LLM. This made it easier to see whether a weak answer came from poor retrieval or from the LLM’s response.

The retrieval worked well for several questions. Q1 retrieved a REST definition from the FastAPI course material. Q2 retrieved the two details needed for the answer: Pydantic-based validation from one document and Uvicorn as the ASGI server from another. Q3 retrieved FastAPI information from two documents. Q4 retrieved different uses of the word “model,” including a Python class representing a database table and a large language model. For Q5 and Q6, the retrieved chunks did not contain relevant evidence, which was expected because the answers were not in the document collection.

The **No RAG** version answered from the model without receiving any retrieved context. It gave reasonable general answers for Q1 through Q3, but it did not follow the document-only requirement for the missing-information questions. For Q4, it interpreted “session” as a class meeting instead of the technical web-session meaning used in the course materials. For Q5, it answered with a typical soccer team size, and for Q6, it gave a general statement about the uncertainty of Bitcoin prices.

Basic RAG improved the answers by giving the retrieved chunks to the model. It recognized that Q5 and Q6 could not be answered from the available context. However, it did not require citations, and it sometimes added details that were not directly stated in the retrieved chunks. For example, the Q2 answer included a specific Uvicorn command that was not shown in the retrieved text.

ContextRAG used labeled sources, source-number citations, and a rule to refuse when evidence was insufficient. It correctly refused Q5 and Q6 using the required sentence. It also handled Q4 by distinguishing the two supported meanings of “model.” However, Q1 showed that the

grounding prompt was not perfect because the answer included a few REST details that were not visible in the retrieved chunks.

The Q2 context-size sweep showed that $k = 3$ was the most useful setting. With $k = 1$, the retrieved chunk covered Uvicorn but not Pydantic validation. With $k = 3$, both needed details were available. With $k = 5$, extra installation and architecture chunks were added without improving the answer.